# A Short Guide to Retrieval-Augmented Generation

*Generic sample document for testing document loaders and RAG pipelines.*

## What is RAG?

Retrieval-Augmented Generation (RAG) combines a retriever with a large language model. Instead of answering from its training data alone, the model receives relevant passages from a knowledge base as context, which reduces hallucinations and lets the system answer questions about private or recent data.

## Document Loading

A RAG pipeline starts by loading source documents such as PDFs, web pages or text files. Loaders turn each source into documents with text content and metadata, for example the file name and page number.

## Chunking

Long documents are split into smaller chunks before indexing. Chunk size and overlap matter: chunks that are too small lose context, while chunks that are too large mix unrelated topics. Recursive character splitting is a strong default, and semantic chunking splits text where the meaning changes.

## Embeddings and Vector Stores

Each chunk is converted into an embedding, a vector that captures its meaning. Embeddings are stored in a vector database such as Chroma, Qdrant or PostgreSQL with pgvector. At query time the question is embedded and the most similar chunks are returned, usually ranked by cosine similarity.

## Advanced Retrieval

Hybrid search combines keyword search (BM25) with vector search, which helps with exact names and identifiers. Multi-query retrieval asks an LLM to rephrase the question several ways. Parent-document retrieval searches small chunks but returns their larger parent sections. Contextual compression removes irrelevant sentences from retrieved chunks.

## Production Concerns

Production RAG systems need tracing and evaluation, for example with LangSmith, plus cost controls such as semantic caching, model routing and token budgets. Answers should cite their sources so users can verify them.